Problem Set 4: Functions and Modules

Prof. Jordan C. Hanson

1 Exercise 1: Comprehensions and Nested Dictionaries

Suppose we want to search the complex plane over the integer coordinates

$$-3 \leq a \leq 3, \qquad -3 \leq b \leq 3,$$

where

$$z = a + bi.$$

1. Use a **list comprehension** to construct all possible pairs $[a, b]$:

```
points = [
    [a, b]
    for a in range(-3, 4)
    for b in range(-3, 4)
]
```

   How many points should appear in the list?

2. Use a **dictionary comprehension** to assign an integer ID to every point:

```
complex_grid = {
    i: {
        "real": points[i][0],
        "imag": points[i][1]
    }
    for i in range(len(points))
}
```

   Each value in `complex_grid` is itself a dictionary. Explain why `complex_grid` is therefore called a **nested dictionary**.

3. Write Python statements that print the real and imaginary parts associated with key `10`.
4. Use a loop to print every entry in the form

```
ID: 10  z = -2 + 0i
```

   Your exact numerical value for key `10` will depend on the order generated by the comprehension.

2 Exercise 2: Complete the `ComplexNumber` Class

Create a Python file named

`complex_number.py`

Define the class

```
class ComplexNumber:
```

Each instance must store

```
self.real
self.imag
```

and use an initializer of the form

```
def __init__(self, real, imag):
    ...
```

Implement the following methods.

Addition

For

$$z_1 = a + bi, \qquad z_2 = c + di,$$

$$z_1 + z_2 = (a + c) + (b + d)i.$$

Implement

```
def __add__(self, other):
    ...
```

so that

```
z3 = z1 + z2
```

returns a new `ComplexNumber` object.

Subtraction

Use

$$z_1 - z_2 = (a - c) + (b - d)i$$

to implement

```
def __sub__(self, other):
    ...
```

Multiplication

Complex multiplication is

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i.$$

Implement

```
def __mul__(self, other):
    ...
```

Complex Conjugate

The complex conjugate of

$$z = a + bi$$

is

$$z^* = a - bi.$$

Implement

```
def conjugate(self):
    ...
```

The method must return a new `ComplexNumber`.

Division

Using the conjugate,

$$\frac{a+bi}{c+di} = \frac{ac+bd}{c^2+d^2} + \frac{bc-ad}{c^2+d^2}i.$$

Implement

```
def __truediv__(self, other):
    ...
```

If the denominator is zero, return `None`.

Display

Finally, implement

```
def __str__(self):
    ...
```

so that

```
z = ComplexNumber(3, -2)
print(z)
```

produces a readable representation such as

```
3 - 2i
```

Your completed class must contain a docstring describing its purpose.

Exercise 3: Finding Poles with the Class

A rational function

$$f(z) = \frac{N(z)}{D(z)}$$

has a **pole** wherever its denominator is zero:

$$D(z) = 0.$$

Create a second Python file named

```
find_poles.py
```

Import your class using

```
from complex_number import ComplexNumber
```

Consider

$$f(z) = \frac{1}{z^2 + 2z + 5}.$$

We will search the integer complex grid from Exercise 1 to find values of z for which

$$D(z) = z^2 + 2z + 5 = 0.$$

Part A: Build the Candidates

Use a list comprehension to create `ComplexNumber` objects:

```
candidates = [
    ComplexNumber(a, b)
    for a in range(-3, 4)
    for b in range(-3, 4)
]
```

Part B: Define the Denominator

Write

```
def denominator(z):
```

that computes

$$z^2 + 2z + 5$$

using only `ComplexNumber` objects and the overloaded operators from your class.

For example, define

```
two = ComplexNumber(2, 0)
five = ComplexNumber(5, 0)
```

and construct the expression from these objects.

Part C: Search for Poles

Loop through `candidates`. For each candidate,

```
value = denominator(z)
```

A pole has been found when

```
value.real == 0 and \
value.imag == 0
```

Store every pole that you find in a list named

```
poles
```

and print the results.

Verify mathematically that your program should find

$$\boxed{z = -1 + 2i}$$

and

$$\boxed{z = -1 - 2i}.$$

Part D: Store the Results

Create a dictionary using a dictionary comprehension in which

- the key is the pole number $1, 2, \ldots$, and
- the value is a nested dictionary containing the real and imaginary parts.

  The structure should have the form

```
{
 1: {"real": -1, "imag":  2},
 2: {"real": -1, "imag": -2}
}
```

Do not type the pole values directly into the dictionary. Construct the dictionary from the list `poles` produced by your search.

Files to Submit

Submit exactly the following three files:

```
complex_number.py
find_poles.py
problem_set_6.pdf
```

The PDF should contain your written answers, mathematical work, and a short explanation of how your pole-search algorithm works.